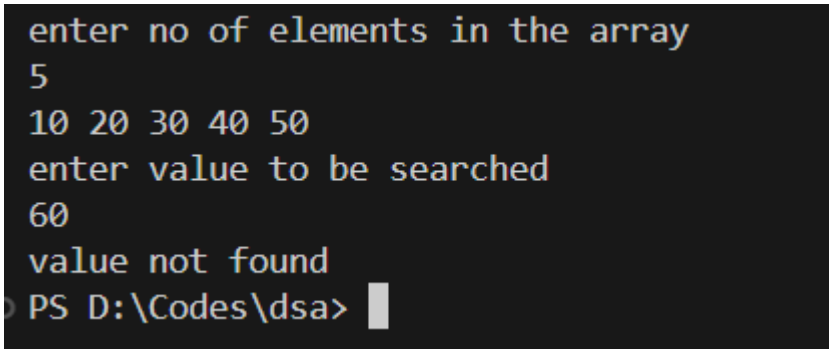
LINEAR SEARCH

```
#include <stdio.h>
int main(){
   int n;
   printf("enter no of elements in the array\n");
   scanf("%d", &n);
   int a[100];
   for(int i = 0; i < n; i++)
      scanf("%d", &a[i]);
   int c = 0;
   int val;
   printf("enter value to be searched\n");
   scanf("%d", &val);
   for(int i = 0; i < n; i++)
      if(a[i] == val)
      {
         c++;
         printf("value found at index = %d", i);
      }
   if(c == 0)
      printf("value not found");
}
```

```
enter no of elements in the array
5
12 43 53 23 37
enter value to be searched
53
value found at index = 2
PS D:\Codes\dsa>
```



BINARY SEARCH – RECURSION

```
#include <stdio.h>
int binarySearch(int a[], int l, int u, int val){
   // u - upper index, l - lower index, val - value to be searched
   int mid = (u + l) / 2;
   if(val == a[mid])
      return mid;
   else if(u < l)
      return -1;
   else if (val > a[mid] )
      binarySearch(a, mid + 1, u, val);
   else if(val < a[mid])
      binarySearch(a, l, mid - 1, val);
}
int main() {
   int n;
   printf("enter no of elements in the array\n");
   scanf("%d", &n);
   printf("enter the elements\n");
   int a[100];
   for(int i = 0; i < n; i++)
      scanf("%d", &a[i]);
   int val;
```

```
    printf("enter value to be searched\n");
    scanf("%d", &val);
    int ans = binarySearch(a, n-1, 0, val);
    if(ans == -1)
        printf("element not found");
    else
        printf("element found at index = %d", ans);
}
```

```
enter no of elements in the array
5
enter the elements
10 20 30 40 50
enter value to be searched
50
element found at index = 4
PS D:\Codes\dsa>
```

```
enter no of elements in the array
5
enter the elements
10 20 30 40 50
enter value to be searched
100
element not found
PS D:\Codes\dsa>
```

```
enter no of elements in the array
5
enter the elements
10 20 30  40 50
enter value to be searched
8
element not found
PS D:\Codes\dsa>
```

BINARY SEARCH – ITERATION

```
#include <stdio.h>

int binarySearch(int a[], int n, int val) {
   int l = 0;
   int u = n - 1;
   while (l <= u) {
      int mid = (u + l) / 2;
      if (val == a[mid]) {
         return mid;
      }
      else if (val > a[mid]) {
         l = mid + 1;
      }
      else {
         u = mid - 1;
      }
   }
   return -1;
}

int main() {
   int n;
   printf("Name:Niveditha \nReg No:24BCE2000 \n");
   printf("enter no of elements in the array\n");
   scanf("%d", &n);
   printf("enter the elements\n");
   int a[100];
   for(int i = 0; i < n; i++)
      scanf("%d", &a[i]);
   int val;
```

```
    printf("enter value to be searched\n");
    scanf("%d", &val);
    int ans = binarySearch(a, n, val);
    if(ans == -1)
        printf("element not found");
    else
        printf("element found at index = %d", ans);
    return 0;
}
```

```
Name:Niveditha
Reg No:24BCE2000
enter no of elements in the array
6
enter the elements
10 29 30 40 50 60
enter value to be searched
50
element found at index = 4
PS E:\Codes\dsa_codes>
```

```
Name:Niveditha
Reg No:24BCE2000
enter no of elements in the array
5
enter the elements
10 20 30 40 50
enter value to be searched
80
element not found
PS E:\Codes\dsa_codes>
```

INSERTION SORT

```
#include <stdio.h>
void print(int arr[], int n){
for(int i = 0; i < n; i++)
printf("%d ", arr[i]);
printf("\n");
}
void insertion(int arr[], int n)
{
for(int j = 1; j < n; j++)
{
int key = arr[j];
int i = j - 1;
while(i >= 0 && key < arr[i])
{
arr[i+1] = arr[i];
printf("j = %d, i = %d, array in inner loop= ", j, i);
print(arr, n);
i --;
}
arr[i+1] = key;
printf("array after every outer loop = ");
print(arr, n);
}
printf("final sorted array = ");
print(arr, n);
}

int main(){
int a[] = {12, 34, 32, 13, 876, 987, 533, 324, 78, 19, 87, 102};
insertion(a, 12);
return 0;
```

}

```
array after every outer loop = 12 34 32 13 876 987 533 324 78 19 87 102
j = 2, i = 1, array in inner loop= 12 34 34 13 876 987 533 324 78 19 87 102
array after every outer loop = 12 32 34 13 876 987 533 324 78 19 87 102
j = 3, i = 2, array in inner loop= 12 32 34 34 876 987 533 324 78 19 87 102
j = 3, i = 1, array in inner loop= 12 32 32 34 876 987 533 324 78 19 87 102
array after every outer loop = 12 13 32 34 876 987 533 324 78 19 87 102
array after every outer loop = 12 13 32 34 876 987 533 324 78 19 87 102
array after every outer loop = 12 13 32 34 876 987 533 324 78 19 87 102
j = 6, i = 5, array in inner loop= 12 13 32 34 876 987 987 324 78 19 87 102
j = 6, i = 4, array in inner loop= 12 13 32 34 876 876 987 324 78 19 87 102
array after every outer loop = 12 13 32 34 533 876 987 324 78 19 87 102
j = 7, i = 6, array in inner loop= 12 13 32 34 533 876 987 987 78 19 87 102
j = 7, i = 5, array in inner loop= 12 13 32 34 533 876 876 987 78 19 87 102
j = 7, i = 4, array in inner loop= 12 13 32 34 533 533 876 987 78 19 87 102
array after every outer loop = 12 13 32 34 324 533 876 987 78 19 87 102
```

```
array after every outer loop = 12 13 19 32 34 78 324 533 876 987 87 102
j = 10, i = 9, array in inner loop= 12 13 19 32 34 78 324 533 876 987 987 102
j = 10, i = 8, array in inner loop= 12 13 19 32 34 78 324 533 876 876 987 102
j = 10, i = 7, array in inner loop= 12 13 19 32 34 78 324 533 533 876 987 102
j = 10, i = 6, array in inner loop= 12 13 19 32 34 78 324 324 533 876 987 102
array after every outer loop = 12 13 19 32 34 78 87 324 533 876 987 102
j = 11, i = 10, array in inner loop= 12 13 19 32 34 78 87 324 533 876 987 987
j = 11, i = 9, array in inner loop= 12 13 19 32 34 78 87 324 533 876 876 987
j = 11, i = 8, array in inner loop= 12 13 19 32 34 78 87 324 533 533 876 987
j = 11, i = 7, array in inner loop= 12 13 19 32 34 78 87 324 324 533 876 987
array after every outer loop = 12 13 19 32 34 78 87 102 324 533 876 987
final sorted array = 12 13 19 32 34 78 87 102 324 533 876 987
```

BUBBLE SORT

```
#include <stdio.h>


void print(int arr[], int n){
for(int i = 0; i < n; i++)
printf("%d ", arr[i]);
printf("\n");
}


void swap(int* a, int* b){
int temp = *a;
*a = *b;
*b = temp;
```

```
}

void bubble(int arr[], int n){
for(int i = 0; i < n - 1; i++) {
for(int j = 0; j < n-i-1; j++)
if(arr[j] > arr[j+1])
{
swap(&arr[j], &arr[j+1]);
printf("i = %d j = %d, array in inner loop after every swap= ", i, j);
print(arr, n);
}
}
printf("final array after sorting = ");
print(arr, n);
}

int main(){
int a[] = {12, 34, 32, 13, 876, 987, 533, 324, 78, 19, 87, 102};
bubble(a, 12);
return 0;
}
```

```
i = 0 j = 1, array in inner loop after every swap= 12 32 34 13 876 987 533 324 78 19 87 102
i = 0 j = 2, array in inner loop after every swap= 12 32 13 34 876 987 533 324 78 19 87 102
i = 0 j = 5, array in inner loop after every swap= 12 32 13 34 876 533 987 324 78 19 87 102
i = 0 j = 6, array in inner loop after every swap= 12 32 13 34 876 533 324 987 78 19 87 102
i = 0 j = 7, array in inner loop after every swap= 12 32 13 34 876 533 324 78 987 19 87 102
i = 0 j = 8, array in inner loop after every swap= 12 32 13 34 876 533 324 78 19 987 87 102
i = 0 j = 9, array in inner loop after every swap= 12 32 13 34 876 533 324 78 19 87 987 102
i = 0 j = 10, array in inner loop after every swap= 12 32 13 34 876 533 324 78 19 87 102 987
i = 1 j = 1, array in inner loop after every swap= 12 13 32 34 876 533 324 78 19 87 102 987
i = 1 j = 4, array in inner loop after every swap= 12 13 32 34 533 876 324 78 19 87 102 987
i = 1 j = 5, array in inner loop after every swap= 12 13 32 34 533 324 876 78 19 87 102 987
i = 1 j = 6, array in inner loop after every swap= 12 13 32 34 533 324 78 876 19 87 102 987
i = 1 j = 7, array in inner loop after every swap= 12 13 32 34 533 324 78 19 876 87 102 987
i = 1 j = 8, array in inner loop after every swap= 12 13 32 34 533 324 78 19 87 876 102 987
i = 1 j = 9, array in inner loop after every swap= 12 13 32 34 533 324 78 19 87 102 876 987
i = 2 j = 4, array in inner loop after every swap= 12 13 32 34 324 533 78 19 87 102 876 987
i = 2 j = 5, array in inner loop after every swap= 12 13 32 34 324 78 533 19 87 102 876 987
i = 2 j = 6, array in inner loop after every swap= 12 13 32 34 324 78 19 533 87 102 876 987
i = 2 j = 7, array in inner loop after every swap= 12 13 32 34 324 78 19 87 533 102 876 987
i = 2 j = 8, array in inner loop after every swap= 12 13 32 34 324 78 19 87 102 533 876 987
i = 3 j = 4, array in inner loop after every swap= 12 13 32 34 78 324 19 87 102 533 876 987
i = 3 j = 5, array in inner loop after every swap= 12 13 32 34 78 19 324 87 102 533 876 987
i = 3 j = 6, array in inner loop after every swap= 12 13 32 34 78 19 87 324 102 533 876 987
i = 3 j = 7, array in inner loop after every swap= 12 13 32 34 78 19 87 102 324 533 876 987
i = 4 j = 4, array in inner loop after every swap= 12 13 32 34 19 78 87 102 324 533 876 987
i = 5 j = 3, array in inner loop after every swap= 12 13 32 19 34 78 87 102 324 533 876 987
i = 6 j = 2, array in inner loop after every swap= 12 13 19 32 34 78 87 102 324 533 876 987
final array after sorting = 12 13 19 32 34 78 87 102 324 533 876 987
PS D:\Codes\dsa>
```

SELECTION SORT

```
#include <stdio.h>

void print(int arr[], int n){

for(int i = 0; i < n; i++)

printf("%d ", arr[i]);

printf("\n");

}


void swap(int* a, int* b){

int temp = *a;

*a = *b;

*b = temp;

}


void selection(int arr[], int n){
```

```
for(int i = 0; i < n; i++)
{
int min = i;
for(int j = i + 1; j < n; j++)
if(arr[j] < arr[min])
{
min = j;
printf("i = %d j = %d min = %d \n", i, j, min);
}
swap(&arr[min], &arr[i]);
printf("array after every outer loop = ");
print(arr, n);
}
printf("final array after sorting = ");
print(arr, n);
}
int main(){
int a[] = {12, 34, 32, 13, 876, 987, 533, 324, 78, 19, 87, 102};
selection(a, 12);
return 0;
}
```

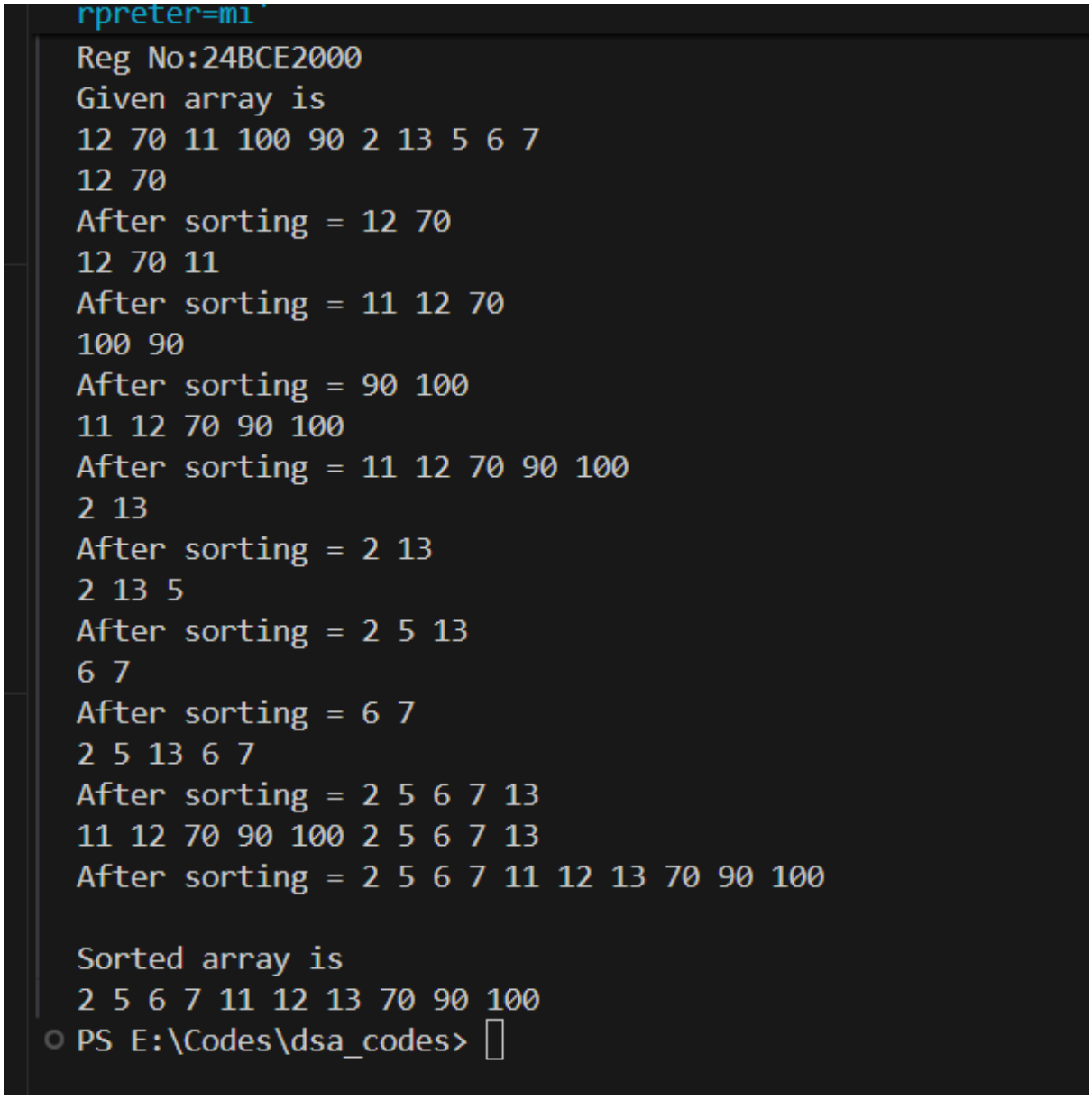
```
array after every outer loop = 12 34 32 13 876 987 533 324 78 19 87 102
i = 1 j = 2 min = 2
i = 1 j = 3 min = 3
array after every outer loop = 12 13 32 34 876 987 533 324 78 19 87 102
i = 2 j = 9 min = 9
array after every outer loop = 12 13 19 34 876 987 533 324 78 32 87 102
i = 3 j = 9 min = 9
array after every outer loop = 12 13 19 32 876 987 533 324 78 34 87 102
i = 4 j = 6 min = 6
i = 4 j = 7 min = 7
i = 4 j = 8 min = 8
i = 4 j = 9 min = 9
array after every outer loop = 12 13 19 32 34 987 533 324 78 876 87 102
i = 5 j = 6 min = 6
i = 5 j = 7 min = 7
i = 5 j = 8 min = 8
array after every outer loop = 12 13 19 32 34 78 533 324 987 876 87 102
i = 6 j = 7 min = 7
i = 6 j = 10 min = 10
array after every outer loop = 12 13 19 32 34 78 87 324 987 876 533 102
i = 7 j = 11 min = 11
array after every outer loop = 12 13 19 32 34 78 87 102 987 876 533 324
i = 8 j = 9 min = 9
i = 8 j = 10 min = 10
i = 8 j = 11 min = 11
array after every outer loop = 12 13 19 32 34 78 87 102 324 876 533 987
i = 9 j = 10 min = 10
array after every outer loop = 12 13 19 32 34 78 87 102 324 533 876 987
array after every outer loop = 12 13 19 32 34 78 87 102 324 533 876 987
array after every outer loop = 12 13 19 32 34 78 87 102 324 533 876 987
final array after sorting = 12 13 19 32 34 78 87 102 324 533 876 987
PS D:\Codes\dsa>
```

MERGE SORT

```
#include <stdio.h>
#include <stdlib.h>
void printArray(int A[], int l1, int r1)
{
   int i;
   for (i = l1; i <= r1; i++)
      printf("%d ", A[i]);
   printf("\n");
}
```

```
void merge(int arr[], int l, int m, int r)
{
    printArray(arr, l, r);
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];
    i = 0;
    j = 0;
    k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    while (j < n2) {
```

```
        arr[k] = R[j];
        j++;
        k++;
    }
    printf("After sorting = ");
    printArray(arr, l, r);
}

void mergeSort(int arr[], int l, int r)
{
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}
int main()
{
    printf("Name:Niveditha \nReg No:24BCE2000 \n");
    int arr[] = { 12, 70, 11, 100, 90, 2, 13, 5, 6, 7 };
    int arr_size = sizeof(arr) / sizeof(arr[0]);
    printf("Given array is \n");
    printArray(arr, 0, arr_size-1);
    mergeSort(arr, 0, arr_size - 1);
    printf("\nSorted array is \n");
    printArray(arr, 0, arr_size-1);
    return 0;
}
```

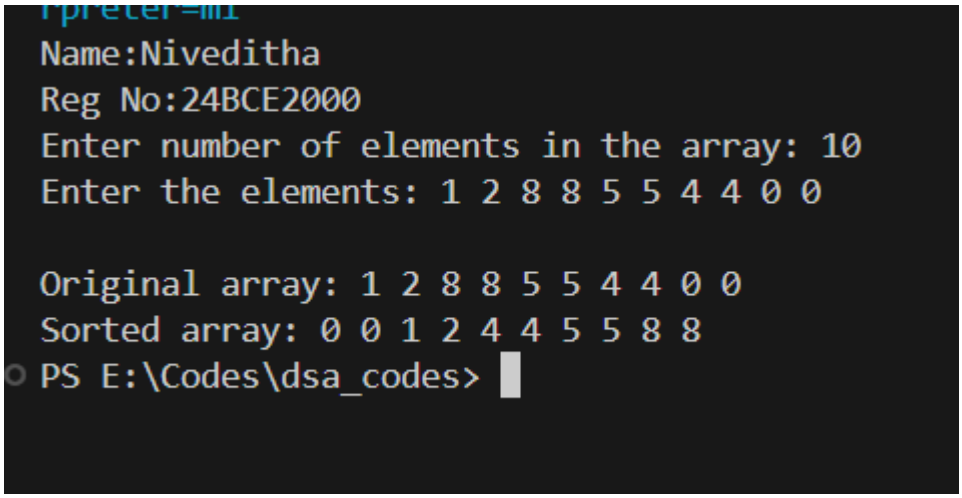


COUNTING SORT

```
#include <stdio.h>


void countingSort(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
```

```
    int count[max + 1];
    for (int i = 0; i <= max; i++) {
        count[i] = 0;
    }

    for (int i = 0; i < n; i++) {
        count[arr[i]]++;
    }

    for (int i = 1; i <= max; i++) {
        count[i] += count[i - 1];
    }

    int output[n];
    for (int i = n - 1; i >= 0; i--) {
        output[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }

    for (int i = 0; i < n; i++) {
        arr[i] = output[i];
    }
}

int main() {
    int n;
    printf("Name:Niveditha \nReg No:24BCE2000 \n");
    printf("Enter number of elements in the array: ");
    scanf("%d", &n);
    int arr[100];
```

```
    printf("Enter the elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    printf("\nOriginal array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    countingSort(arr, n);
    printf("\nSorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

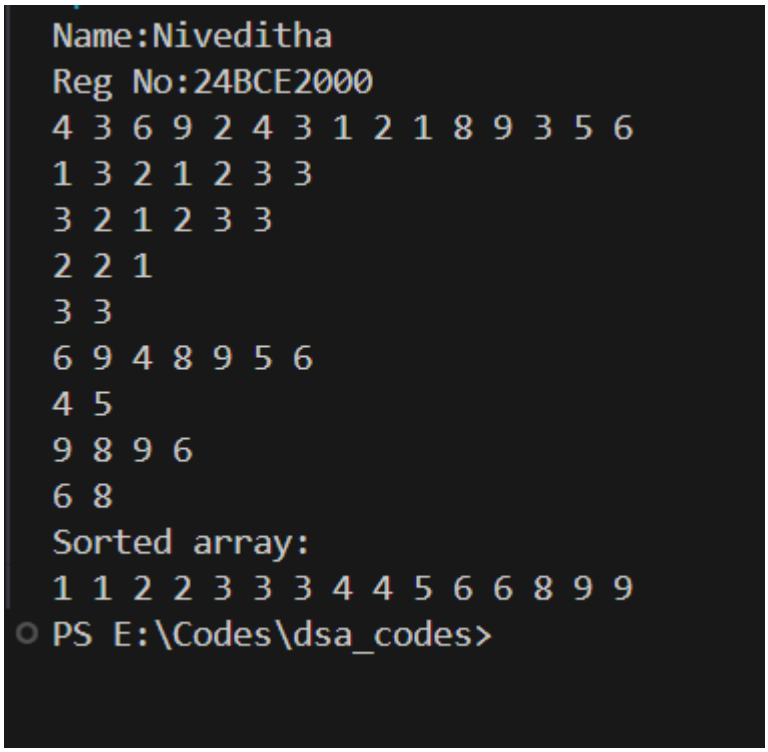


QUICK SORT

```
#include <stdio.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
```

```
}
void printArray(int arr[], int start, int stop)
{
   for (int i = start; i <= stop; i++)
      printf("%d ", arr[i]);
   printf("\n");
}

int partition(int arr[], int low, int high) {
   printArray(arr, low, high);
   int pivot = arr[low];
   int start = low;
   int end = high;
   int k = high;
   for (int i = high; i > low; i--) {
      if (arr[i] >= pivot)
         swap(&arr[i], &arr[k--]);
   }
   swap(&arr[low], &arr[k]);
   return k;
}

void quickSort(int arr[], int low, int high) {
   if (low < high) {
      int idx = partition(arr, low, high);
      quickSort(arr, low, idx - 1);
      quickSort(arr, idx + 1, high);
   }
}

int main() {
```

```
    printf("Name:Niveditha \nReg No:24BCE2000 \n");
    int arr[] = {4, 3, 6, 9, 2, 4, 3, 1, 2, 1, 8, 9, 3, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);
    quickSort(arr, 0, n - 1);
    printf("Sorted array: \n");
    printArray(arr, 0, n-1);
    return 0;
}
```



```
rpreter=m1'
Reg No:24BCE2000
Given array is
12 70 11 100 90 2 13 5 6 7
12 70
After sorting = 12 70
12 70 11
After sorting = 11 12 70
100 90
After sorting = 90 100
11 12 70 90 100
After sorting = 11 12 70 90 100
2 13
After sorting = 2 13
2 13 5
After sorting = 2 5 13
6 7
After sorting = 6 7
2 5 13 6 7
After sorting = 2 5 6 7 13
11 12 70 90 100 2 5 6 7 13
After sorting = 2 5 6 7 11 12 13 70 90 100

Sorted array is
2 5 6 7 11 12 13 70 90 100
PS E:\Codes\dsa_codes>
```